Chiticontrol
-
Mini maşină

Cuprins

I. Project Presentation 4

II. Technologies Used 4

II.1 C++ 4

II.2 Java 5

II.3 Android Studio 5

II.4 ESP32 6

III. Conceptual Scheme 6

IV. Real-Time Testing 8

V. Appendices 10

VI. Bibliography and Copyright 22

I. Project Presentation

In the context of automation and control of mobile devices, an important role is played by the design and implementation of wireless control systems. Both industrial applications, as well as recreational ones, benefit from the possibility of operating robots and vehicles from distance, offering increased flexibility and accessibility.

The present project explores the development of such a system, focusing on the creation of a mobile robot with four wheels, controlled through an Android application. The robot is equipped with a propulsion system on the rear wheels (RWD) and a servomotor for the steering of the front wheels, allowing efficient maneuverability.

The central element of the system is the ESP32 board, a powerful microcontroller that ensures wireless communication through Bluetooth. A distinctive aspect of this project is represented by the implementation from zero of the Bluetooth communication system, both at the level of the Android application (developed in Java), as well as at the level of the ESP32 microcontroller (programmed in C++). To facilitate the reuse of the code and to improve the structure of the project, dedicated classes were created for managing Bluetooth communication: BluetoothManager (Java) and btManager (C++). This approach allowed a fine optimization of data transfer and a precise control over the interaction between the two components. The Android application developed specifically for this project offers an intuitive interface, facilitating the precise control of the robot and offering the user an interactive and accessible experience.

Therefore, my project demonstrates the efficient integration of modern hardware and software technologies, with a special attention given to the customized and modular development of the Bluetooth communication system, in order to achieve a robust and versatile control system for a mobile robot.

II. Technologies Used

II.1 C++

C++ is a mid-level programming language, developed by Bjarne Stroustrup starting with 1979 as an extension of the C language. C++ combines the characteristics of object-oriented programming with the efficiency and control over hardware offered by C. This makes it a powerful and versatile language, used in a variety of fields, including the development of operating systems, games, desktop applications, embedded systems and high-performance software.

C++ supports both procedural programming, as well as object-oriented programming, allowing developers to choose the paradigm that best fits the needs of the project. The language offers direct control over memory management, which can lead to more efficient applications, but requires increased attention from the programmer to avoid errors.

Over time, C++ has been standardized by ISO, resulting in multiple versions (C++98, C++03, C++11, C++14, C++17, C++20, C++23) which brought improvements and new characteristics to the language. C++ remains an essential language for developers who need maximum performance and control over system resources.

II.2 Java

Java is an object-oriented programming language, initially designed by James Gosling at Sun Microsystems (later acquired by Oracle Corporation) in 1995. One of the fundamental principles of Java is portability, illustrated by the slogan “write once, run anywhere” (WORA - Write Once, Run Anywhere). This characteristic is obtained by compiling the Java source code into an intermediate code called bytecode, which is executed by the Java Virtual Machine (JVM).

JVM is a software that abstracts the operating system and the underlying hardware, allowing Java applications to run on any device that has a JVM implementation. Java offers a wide range of functionalities, from the development of desktop and web applications to mobile applications (Android) and enterprise systems.

The Java language is known for its clear syntax, automatic memory management (garbage collection) and rich libraries. It has evolved significantly over the years, adding support for concurrent, generic and functional programming. Java continues to be a popular language in the industry, being widely used in the development of complex and scalable applications.

II.3 Android Studio

Android Studio is an integrated development environment (IDE) official for the development of Android applications. It is based on IntelliJ IDEA, a powerful Java IDE, and offers a complete set of tools for designing, developing, testing and debugging Android applications. Android Studio is developed and maintained by Google.

The IDE includes an intelligent code editor, an emulator to test applications on different virtual Android devices, a profiler to analyze application performance and a flexible build system based on Gradle. Android Studio supports development in the Java and Kotlin languages, both being official languages for Android development.

Android Studio offers a series of characteristics specific to Android development, such as a visual layout editor for designing user interfaces, support for specific Android libraries and tools for managing application resources (images, character strings, etc.). By using Android Studio, developers can create high-quality Android applications, optimized for performance and compatibility with various versions of the operating system.

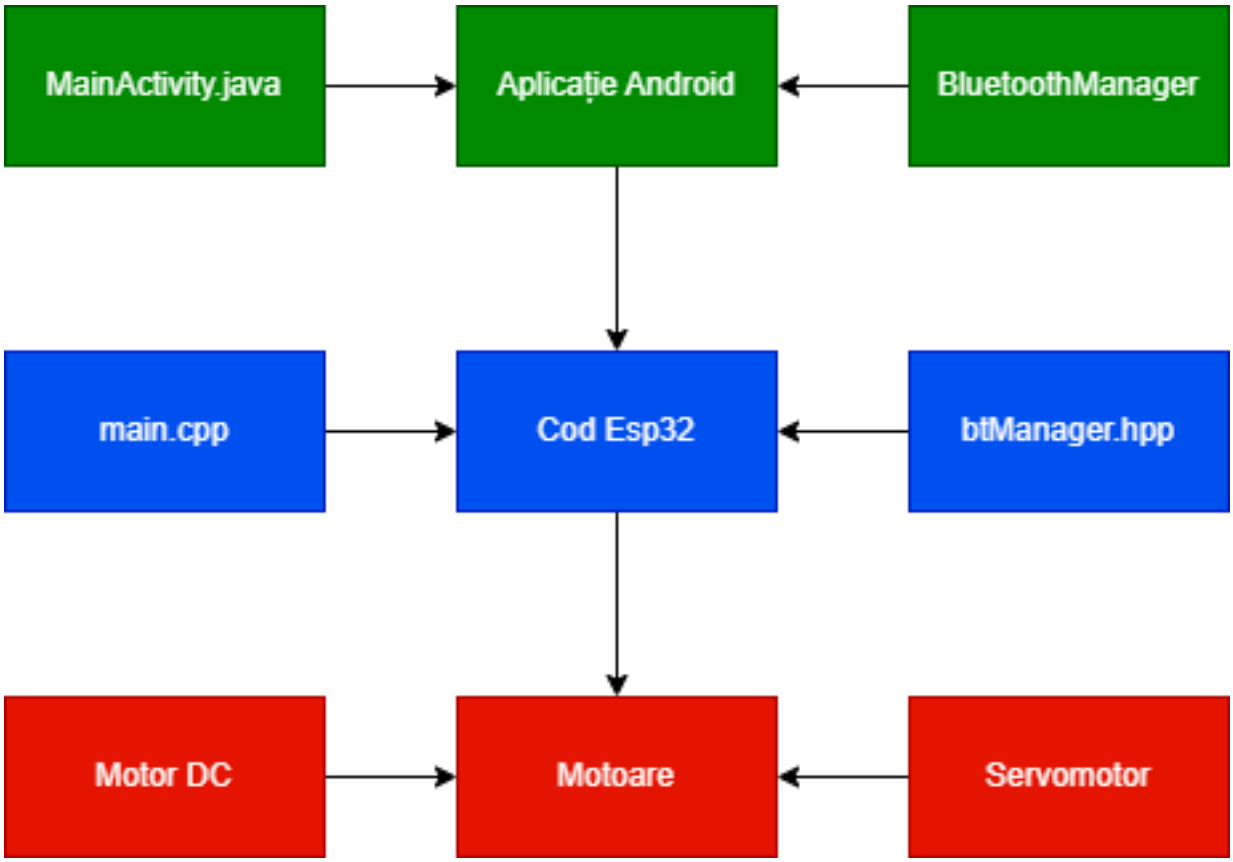
II.4 ESP32

ESP32 is a series of microcontrollers low-cost, low-power, developed by Espressif Systems. These microcontrollers integrate Wi-Fi and Bluetooth connectivity, which makes them ideal for Internet of Things (IoT) applications and embedded projects. ESP32 includes a dual-core or single-core processor, memory, peripherals (GPIO, UART, SPI, I2C, etc.) and an analog-digital converter.

ESP32 is programmable mainly using the C and C++ languages, with the help of the SDK (Software Development Kit) offered by Espressif. There is also support for other languages and platforms, such as MicroPython and Arduino. The popularity of ESP32 is due to its combination of performance, connectivity and low cost, allowing developers to create a wide range of smart and connected devices.

ESP32 is used in various applications, including home automation, wearable devices, wireless sensors, industrial control and many others. The active developer community and the resources available online contribute to the ease of use and the continuous expansion of the capabilities of this microcontroller.

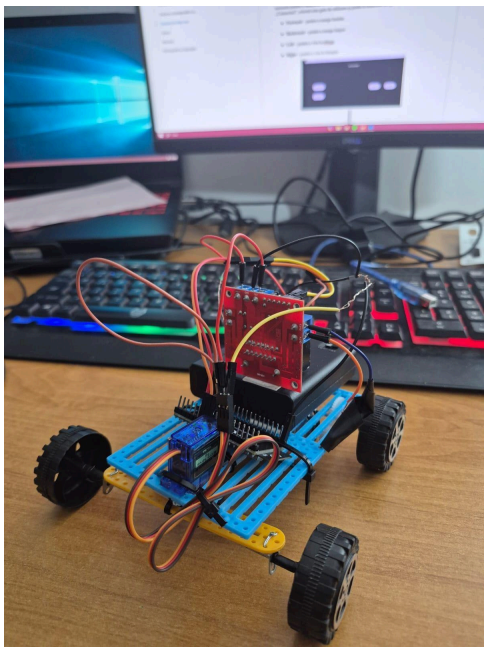
III. Conceptual scheme of the work

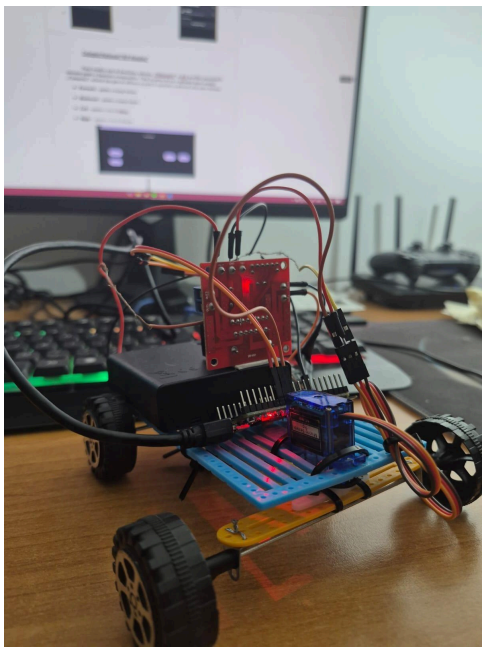


```
.
└── Proiect atestat
    ├── AplicațieAndroid
    │   └── app
    │       └── src
    │           └── main
    │               └── java
    │                   ├── com
    │                   │   └── example
    │                   │       └── myapplication
    │                   │           └── MainActivity.java
    │                   └── custom
    │                       └── BluetoothManager.java
    └── esp32
        ├── include
        │   └── btManager.hpp
        └── src
            └── main.cpp
```

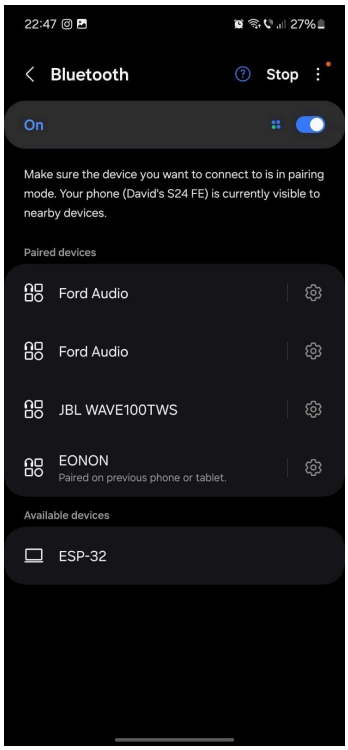
IV. Real-time testing

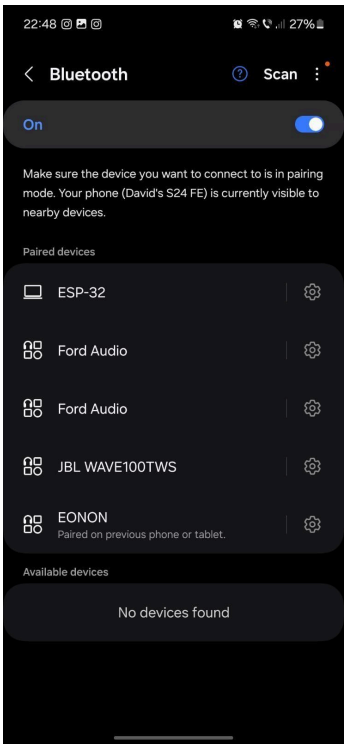
To use the robot, we will have to power the ESP32 board using a microUSB cable and to turn on the battery using the switch located on it. Two red LEDs will light up: one on the ESP32 board and the other on the motor driver, sign that both components function correctly.





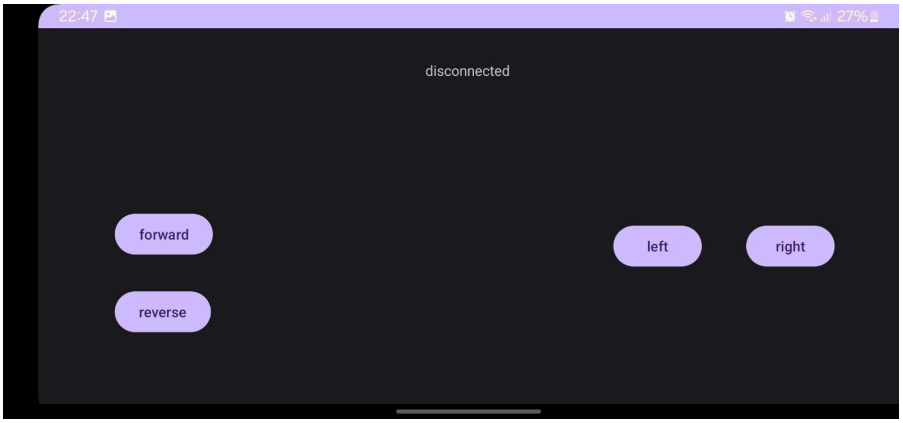
After we make sure that the hardware part is operational, we will activate the Bluetooth of the phone and we will pair it with the device named “ESP-32”.

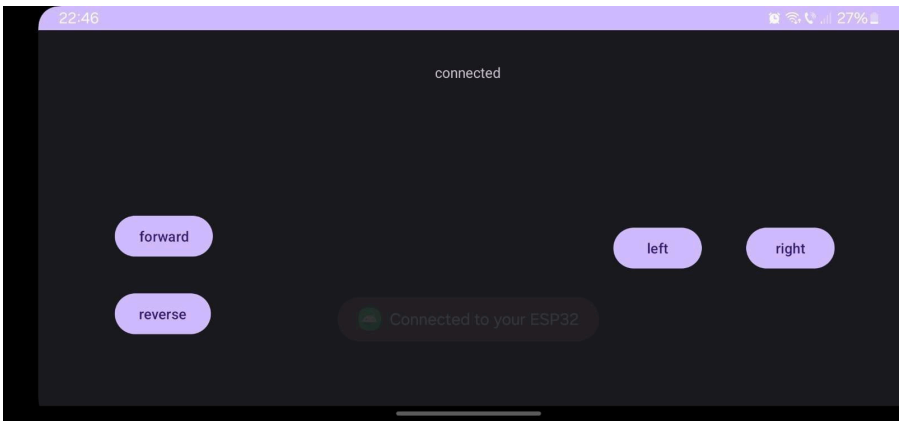




The next step is to open the “Chiticontrol” application and to grant it the necessary permissions to function properly. When in the upper part of the application appears the message “Connected”, the robot is ready for use and can be controlled with the help of the four buttons:

- **Forward – to go forward**
- **Backward – to go backward**
- **Left – to turn left**
- **Right – to turn right**





Demonstration video: https://youtu.be/GVRYfFZE36M?feature=shared

V. Appendices

Project code

- btManager.hpp

```
#include <Arduino.h>
#include <BluetoothSerial.h>
#include <cstring>
BluetoothSerial serialBt;

class btManager{

  public:
    btManager(HardwareSerial& serial) : serial(serial){
      if (!serialBt.begin("ESP-32")) {
        serial.println("An error occurred initializing Bluetooth");
      } else {
        serial.println("Bluetooth initialized");
      }
    }

    void Send(char* message){
      serialBt.println(message);
    }

    void Receieve(char* currentLine) {
      if(serialBt.available()) {
        int character = serialBt.read();
                                                  if (character != '\n' && newLineIndex < 127) {
overflow                                          // Check for newline and buffer

                                                 newLine[newLineInde
                                                 x++] =
                                                 (char)character;
                                                 newLine[newLineInde
                                                 x] = '\0'; //
                                                 Null-terminate the
                                                 string
        } else if (character == '\n') {

          strcpy(currentLine, newLine);
          serial.println(currentLine);
          newLineIndex = 0; // Reset index
          newLine[0] = '\0'; // Clear the string
        }
      }
    }

  private:

    HardwareSerial serial;
    char newLine[128] = "";
```

```
    int newLineIndex = 0;

};
```

- BluetoothManager.java

```
package custom;

import android.Manifest;
import android.app.Activity;
import android.content.pm.PackageManager;
import android.content.Intent;
import android.bluetooth.*;
import android.widget.TextView;
import android.widget.Toast;

import android.os.HandlerThread;
import android.os.Handler;

import androidx.core.content.ContextCompat;
import androidx.core.app.ActivityCompat;

import java.io.*;
import java.util.UUID;
import
android.util.Log;

import com.example.myapplication.R;

public class BluetoothManager {

    HandlerThread ConnectionThread;
    Handler ConnectionHandler;

    HandlerThread ReceiverThread;
    Handler ReceiverHandler;

    private final Activity activity;

    private BluetoothAdapter bluetoothAdapter;
    private BluetoothSocket bluetoothSocket;
    private BluetoothDevice bluetoothDevice;
    private InputStream inputStream;
    private OutputStream outputStream;

                            private       static       final       UUID       MY_UUID       =
UUID.fromString("00001101-0000-1000-8000-00805F9B34FB");
    private static final String esp32_name = "ESP-32";

    private final int PERMISSION_REQUEST_CODE = 1;
    private final int BLUETOOTH_ENABLE_REQUEST = 2;

    private boolean PermissionsGranted = false; // Explicit initialization

    TextView connectionLabel;

    public BluetoothManager(Activity activity) {
        this.activity = activity;
           this.connectionLabel = activity.findViewById(R.id.connectionLabel); // Initialize
here
```

```
ConnectionThread = new HandlerThread("ConnectionThread");
```

```
        ConnectionThread.start();
        ConnectionHandler = new Handler(ConnectionThread.getLooper());

        ReceiverThread = new HandlerThread("ReceiverThread");
        ReceiverThread.start();
        ReceiverHandler = new Handler(ReceiverThread.getLooper());

        checkPermissions();
    }

    public void setPermissionsGranted(boolean value) {
        PermissionsGranted = value;
        if(value) {
            EnableBluetooth();
        }else{
            activity.runOnUiThread(new Runnable() {
                @Override
                public void run() {
                    connectionLabel.setText(R.string.permissionsDeclined);
                }
            });
        }
    }

    public void checkPermissions() {
                                            if     (ContextCompat.checkSelfPermission(activity,
Manifest.permission.BLUETOOTH_CONNECT) != PackageManager.PERMISSION_GRANTED ||
                                                     ContextCompat.checkSelfPermission(activity,
Manifest.permission.BLUETOOTH_SCAN) != PackageManager.PERMISSION_GRANTED) {
            ActivityCompat.requestPermissions(activity,
                                           new String[]{Manifest.permission.BLUETOOTH_CONNECT,
Manifest.permission.BLUETOOTH_SCAN},
                    PERMISSION_REQUEST_CODE);
        } else {
            PermissionsGranted = true;
                         Toast.makeText(activity, "Bluetooth Permissions Already Granted",
Toast.LENGTH_SHORT).show();
            EnableBluetooth(); // Enable bluetooth right away.
        }
    }

    public void EnableBluetooth() {
        bluetoothAdapter = BluetoothAdapter.getDefaultAdapter();

        if (bluetoothAdapter == null) {
            Log.e("Bluetooth", "Bluetooth not supported on this device");
            return;
        }

        // Check if Bluetooth is enabled
        if(!bluetoothAdapter.isEnabled()){
            Intent enableBtIntent = new Intent(BluetoothAdapter.ACTION_REQUEST_ENABLE);
                activity.startActivityForResult(enableBtIntent, BLUETOOTH_ENABLE_REQUEST); //
Request Bluetooth enable
        }

        if(bluetoothAdapter.isEnabled()){
            searchEsp32();
```

```
        }

    }

    private void searchEsp32() {
        ConnectionHandler.post(new Runnable() {
            @Override
            public void run() {
                // First, try to find the ESP32 from paired devices
                bluetoothDevice = getDevice(esp32_name);

                                                if (bluetoothDevice == null) {
                                                        // If the device is not found in paired
delay retrying...");
                                                         devices, retry after a short

                                                         Log.d("BluetoothSearch", "ESP32 Device

                                                         not found in paired devices,

                                                    ConnectionHandler.postDelayed(this, 100);
                                                    // Retry after 3 seconds
                                                } else {
                                                    // Bluetooth device found, create
                                                    connection
                                                    connectToDevice(bluetoothDevice);
                }
            }
        });
    }

    private void connectToDevice(BluetoothDevice device) {
        ConnectionHandler.post(new Runnable() {
            @Override
            public void run() {
                try {
                    bluetoothSocket = device.createRfcommSocketToServiceRecord(MY_UUID);
                    bluetoothSocket.connect(); // Attempt to connect

                    inputStream = bluetoothSocket.getInputStream();
                    outputStream = bluetoothSocket.getOutputStream();

                    Log.d("BluetoothSearch", "Connected to ESP32");
                    activity.runOnUiThread(new Runnable() {
                        @Override
                        public void run() {
                            connectionLabel.setText(R.string.connected);
                                         Toast.makeText(activity, "Connected to your ESP32",
Toast.LENGTH_SHORT).show();
                        }
                    });

                    startReceiving();

                } catch (IOException e) {
                    Log.e("Bluetooth", "Error connecting or communicating with ESP32", e);
                    activity.runOnUiThread(new Runnable() {
                        @Override
                        public void run() {
```

```
        connectionLabel.setText(R.string.disconnected);
    }
});
// Retry if connection failed
Log.d("BluetoothConnection", "Retrying connection in 0.1 seconds...");
```

```
                                                    ConnectionHandler.postDelayed(this, 100);
                                                    // Retry after 1 seconds
                                                }
            }
        });
    }

    // Helper method to get Bluetooth device by name
    private BluetoothDevice getDevice(final String deviceName) {
        for (BluetoothDevice device : bluetoothAdapter.getBondedDevices()) {
            if (device.getName().equals(deviceName)) {
                return device;
            }
        }
        return null;
    }

    //Send
    public void Send(String messageToSend) {
        if(checkConnection()){
            try{
                outputStream.write((messageToSend+"\n").getBytes());
                Log.d("Bluetooth", "Sent: " + messageToSend);
            }catch (IOException e) {
                Log.e("BluetoothSend", "Error sending to ESP32", e);
                      }
        }else{                                  Toast.makeText(activity,  esp32_name  +  "
                                                not  connected",
Toast.LENGTH_SHORT).show();
        }

    }


    //Recieve
    public void processReceived(String receivedData) {
        // Update UI if needed (use runOnUiThread)
        activity.runOnUiThread(() -> {
            Toast.makeText(activity, "Received: " + receivedData, Toast.LENGTH_SHORT).show();

        });
    }

    public void startReceiving() {
        if (inputStream == null) {
            Log.e("Bluetooth", "InputStream is null. Cannot start receive thread.");
            return;
        }

        ReceiverHandler.post(new Runnable() {
            @Override
            public void run() {
                try {
                    byte[] buffer = new byte[1024];
                    int bytes;
                    while (checkConnection() && (bytes = inputStream.read(buffer)) != -1) {
                        String receivedData = new String(buffer, 0, bytes);
                        Log.d("BluetoothDebug", "Received: " + receivedData);
                        processReceived(receivedData);
```

```
    }
} catch (IOException e) {
```

```
                                                    Log.e("Bluetooth", "Error
                                                    receiving data", e);
                                                    searchEsp32(); // Attempt to
            }                                       reconnect if stream is
        });                                         broken
    }                                           }


    public boolean checkConnection(){
        return bluetoothSocket != null && bluetoothSocket.isConnected();
    }

    public void Destroy() {
        try {
            if (inputStream != null) {
                inputStream.close();
            }
            if (outputStream != null) {
                outputStream.close();
            }
            if (bluetoothSocket != null) {
                bluetoothSocket.close();
            }
        } catch (IOException e) {
            Log.e("BluetoothManager", "Error closing resources", e);
        }
    }
}
```

- MainActivity.java

```
package com.example.myapplication;

import custom.BluetoothManager;

import android.content.pm.PackageManager;
import android.os.Bundle;
import android.os.Handler;
import android.os.HandlerThread;
import android.util.Log;
import android.widget.*;
import android.view.View;
import android.view.MotionEvent;
import androidx.appcompat.app.AppCompatActivity;
import android.content.Intent;

public class MainActivity extends AppCompatActivity {

    /*HandlerThread backgroundThread;
    Handler backgroundHandler;
    Runnable backgroundRunnable = new Runnable(){
        @Override
        public void run() {
            if(bt.checkConnection()){
                connectionLabel.setText(R.string.connected);
            }else{
                connectionLabel.setText(R.string.disconnected);
```

}

```
        }
    };*/

    private BluetoothManager bt;
    private Button forward, reverse, left, right;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        /*backgroundThread = new HandlerThread("backgroundThread");
        backgroundThread.start();
        backgroundHandler = new Handler(backgroundThread.getLooper());*/


        bt = new BluetoothManager(this);
        //backgroundHandler.post(backgroundRunnable);
        bt.startReceiving();

        forward = findViewById(R.id.Accelerate);
        reverse = findViewById(R.id.Reverse);
        left = findViewById(R.id.Left);
        right = findViewById(R.id.Right);

        forward.setOnTouchListener(this::Forward_Touch);
        reverse.setOnTouchListener(this::Reverse_Touch);
        left.setOnTouchListener(this::Left_Touch);
        right.setOnTouchListener(this::Right_Touch);
    }

    boolean forwardButtonPressed = false;
    boolean reverseButtonPressed = false;
    boolean leftButtonPressed = false;
    boolean rightButtonPressed = false;

    boolean Forward_Touch(View v, MotionEvent event) {
        switch (event.getAction()) {
            case MotionEvent.ACTION_DOWN:
                forwardButtonPressed = true;
                if (bt.checkConnection()) {
                    bt.Send("forward");
                } else {
                                Toast.makeText(this, "No device connected through bluetooth",
Toast.LENGTH_SHORT).show();
                }
                return true;
            case MotionEvent.ACTION_UP:
                if (forwardButtonPressed) {
                    forwardButtonPressed = false;
                    if (bt.checkConnection()) {
                        bt.Send("stop");
                        Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    }
                }
                return true;//SA FAC ASTA SI LA RESTUL BUTOANELOR*/

            case MotionEvent.ACTION_CANCEL:
```

```
                if (forwardButtonPressed) {
                    forwardButtonPressed = false;
                    Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    if (bt.checkConnection()) {
                        bt.Send("stop");
                        Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    }
                }
                return true;
        }
        return false;
    }

    boolean Reverse_Touch(View v, MotionEvent event) {
        switch (event.getAction()) {
            case MotionEvent.ACTION_DOWN:
                reverseButtonPressed = true;
                if (bt.checkConnection()) {
                    bt.Send("reverse");
                } else {
                                Toast.makeText(this, "No device connected through bluetooth",
Toast.LENGTH_SHORT).show();
                }
                return true;
            case MotionEvent.ACTION_UP:
                if (reverseButtonPressed) {
                    reverseButtonPressed = false;
                    if (bt.checkConnection()) {
                        bt.Send("stop");
                        Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    }
                }
                return true;//SA FAC ASTA SI LA RESTUL BUTOANELOR*/

            case MotionEvent.ACTION_CANCEL:
                if (reverseButtonPressed) {
                    reverseButtonPressed = false;
                    Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    if (bt.checkConnection()) {
                        bt.Send("stop");
                        Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    }
                }
                return true;
        }
        return false;
    }

    boolean Left_Touch(View v, MotionEvent event) {
        switch (event.getAction()) {
            case MotionEvent.ACTION_DOWN:
                leftButtonPressed = true;
                if (bt.checkConnection()) {
                    bt.Send("left");
                } else {
                                Toast.makeText(this, "No device connected through bluetooth",
Toast.LENGTH_SHORT).show();
                }
```

```
                return true;
            case MotionEvent.ACTION_UP:
                if (leftButtonPressed) {
                    leftButtonPressed = false;
                    if (bt.checkConnection()) {
                        bt.Send("stopSteering");
                        //Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    }
                }
                return true;

            case MotionEvent.ACTION_CANCEL:
                if (leftButtonPressed) {
                    leftButtonPressed = false;
                    if (bt.checkConnection()) {
                        bt.Send("stopSteering");
                        Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    }
                }
                return true;
        }
        return false;
    }

    boolean Right_Touch(View v, MotionEvent event) {
        switch (event.getAction()) {
            case MotionEvent.ACTION_DOWN:
                rightButtonPressed = true;
                if (bt.checkConnection()) {
                    bt.Send("right");
                } else {
                                Toast.makeText(this, "No device connected through bluetooth",
Toast.LENGTH_SHORT).show();
                }
                return true;
            case MotionEvent.ACTION_UP:
                if (rightButtonPressed) {
                    rightButtonPressed = false;
                    if (bt.checkConnection()) {
                        bt.Send("stopSteering");
                        Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    }
                }
                return true;

            case MotionEvent.ACTION_CANCEL:
                if (rightButtonPressed) {
                    rightButtonPressed = false;
                    if (bt.checkConnection()) {
                        bt.Send("stopSteering");
                        Toast.makeText(this, "Stopped", Toast.LENGTH_SHORT).show();
                    }
                }
                return true;
        }
        return false;
    }
```

```
    @Override
      public void onRequestPermissionsResult(int requestCode, String[] permissions, int[]
grantResults) {
        super.onRequestPermissionsResult(requestCode, permissions, grantResults);

        if (requestCode == 1) {
            boolean allGranted = true;
            for (int result : grantResults) {
                if (result != PackageManager.PERMISSION_GRANTED) {
                    allGranted = false;
                    break;
                }
            }
            if (allGranted) {
                Log.d("acces", "Permissions granted");
                Toast.makeText(this, "Permissions Granted", Toast.LENGTH_SHORT).show();
                  bt.setPermissionsGranted(true); // Proceed with Bluetooth connection after
permissions are granted
            } else{
                bt.setPermissionsGranted(false);
                Toast.makeText(this, "Permissions Denied", Toast.LENGTH_SHORT).show();
            }
        }
    }

    // Handle Bluetooth enabling result
    @Override
    protected void onActivityResult(int requestCode, int resultCode, Intent data) {
        super.onActivityResult(requestCode, resultCode, data);
        if (requestCode == 2) { // Bluetooth enable request
            if (resultCode == RESULT_OK) {
                 // bt.connectToEsp32(); // Proceed with Bluetooth connection after enabling
Bluetooth                                            } else {
                                                         Toast.makeText(this, "Bluetooth not
                                                         enabled", Toast.LENGTH_SHORT).show();
            }
        }
    }
}
```

- main.cpp

```
#include <btManager.hpp>
#include <ESP32Servo.h>

Servo servo;
int forwardMotor = 12, reverseMotor = 14;
int m = 0, s = 0, steeringAngle;
bool isSteering;
btManager *bt =
nullptr;
char *message = new char[128];

void processMessage(const char* message);
void drive();

void setup() {
```

```
Serial.begin(9600);
```

```
  bt = new btManager(Serial);
  pinMode(forwardMotor, OUTPUT);
  pinMode(reverseMotor, OUTPUT);
  digitalWrite(forwardMotor, LOW);
  digitalWrite(reverseMotor, LOW);
  servo.attach(27);
  steeringAngle = 100;
  servo.write(steeringAngle);
  isSteering = false;
}

void loop() {
  bt->Receieve(message);
  processMessage(message);
  drive();
}

void processMessage(const char* message){
      if(strcmp(message, "forward") == 0) m=1;
      else if(strcmp(message, "reverse") == 0) m=-1;
      else if(strcmp(message, "stop") == 0) m=0;


      if(strcmp(message, "left") == 0){
        s=-1;
        isSteering = true;
      }
      else if(strcmp(message, "right") == 0){
        s=1;
        isSteering = true;
      }
      else if(strcmp(message, "stopSteering") == 0){
        s=0;
        isSteering = true;
      }

    }

void drive(){
  switch(m){
    case 1:
      digitalWrite(forwardMotor, HIGH);
      digitalWrite(reverseMotor, LOW);
      break;
    case -1:
      digitalWrite(forwardMotor, LOW);
      digitalWrite(reverseMotor, HIGH);
      break;
    default:
      digitalWrite(forwardMotor, LOW);
      digitalWrite(reverseMotor, LOW);
      break;
  }

  if(isSteering){
    switch(s){
    case 1:
      steeringAngle = 120;
```

```
      break;
    case -1:
      steeringAngle = 80;
      break;
    default: steeringAngle
      = 100; break;
    }
    servo.write(steeringAngle);
    isSteering = false;
  }

}
```

VI. Bibliography and Copyright

- https://en.wikipedia.org/wiki/Java_(programming_language)
- https://en.wikipedia.org/wiki/C%2B%2B
- https://en.cppreference.com/w/
- https://en.wikipedia.org/wiki/Android_Studio
- https://developer.android.com/studio
- https://randomnerdtutorials.com/